

Optimisation Code_Aster 2024 — Analyse et Propositions

Projet CGCAL — Architecture RAILWAY (ASSE) Fichiers analysés :
railway_combination_inst.py, railway_unit_cases.py, templates phase 7/9/11
Date : 2026-05-20 | Statut : **EN ATTENTE D'APPROBATION** — rien n'a été modifié

ÉTAPE 2 — Analyse Comparative

Aspect	Ancienne architecture	Nouvelle architecture (code actuel)
Nb MECA_STATIQUE (combinaisons)	$N_{cas} \times N_{familles}$ (ex: $30 \times 4 = 120$ appels)	0 — remplacé par CREA_CHAMP ASSE
Nb MECA_STATIQUE (total)	$120 + N_{UC}$	$N_{UC} + N_{ELUSIS}$ ($\approx 15 + 48 = 63$)
Nb CALC_CHAMP (phase 11)	$N_{cas} \times 3$ familles	$N_{cas} \times 3$ familles (inchangé)
CREA_CHAMP	EXTR seulement	ASSE + EXTR (superposition linéaire)
GESTION_MEMOIRE solveur	Non spécifié	'IN_CORE' — forcé RAM → à corriger
NB_RHS	Absent	$\min(N, 32)$ — présent (ELUSIS + ASSE)
RENUM	Absent	'METIS' — présent (optimal DKT)
PCENT_PIVOT	Absent	20 — présent (bon réglage)
DETRUIRE	Absent	Absent — opportunité majeure
Code Python factorisé	Répétition manuelle	Boucles Python — bien factorisé

Observations critiques

- GESTION_MEMOIRE='IN_CORE' est présent à **3 endroits** (unit cases, seismic ELUSIS, fonction legacy). Pour un maillage DKT avec ~30 cas unitaires, la factorisation MUMPS peut consommer plusieurs GB. 'IN_CORE' interdit tout recours au swap — risque de crash OOM sur Windows avec RAM limitée.

- Aucun DETRUIRE dans toute la chaîne : les champs `deplcomb{i}*`, `fcal*`, `DEPL*`, `sig*` (individuels par cas) restent tous en base jusqu'à la fin de phase 11, alors qu'ils sont inutiles dès leur consommation dans `CREA_RESU`.
 - La **fonction** `build_railway_results_block` (lignes 464–550) génère des `MECA_STATIQUE` par combinaison (ancienne architecture) — elle n'est **plus appelée** mais reste dans le fichier. Laissée intacte.
-

ÉTAPE 3 / 4 — Optimisations Proposées

OPTIMISATION 1 — `GESTION_MEMOIRE='AUTO'` dans les cas unitaires

FICHER : `aster/comm_blocks/railway_unit_cases.py`, ligne 128

AVANT :

```
SOLVEUR=_F(  
    GESTION_MEMOIRE='IN_CORE',  
    METHODE='MUMPS',  
    NPREC=6,  
    PCENT_PIVOT=20, # OPT: default 10-20 sufficient for linear elastic  
                   # shells; 40 causes excess fill-in  
    RENUM='METIS', # OPT: METIS reduces fill-in 30-50% vs AUTO/MD on  
                  # unstructured DKT meshes  
    RESI_RELA=1e-03  
)
```

APRÈS :

```
SOLVEUR=_F(  
    GESTION_MEMOIRE='AUTO',  
    METHODE='MUMPS',  
    NPREC=6,  
    PCENT_PIVOT=20, # OPT: default 10-20 sufficient for linear elastic  
                   # shells; 40 causes excess fill-in  
    RENUM='METIS', # OPT: METIS reduces fill-in 30-50% vs AUTO/MD on  
                  # unstructured DKT meshes  
    RESI_RELA=1e-03  
)
```

GAIN ESTIMÉ : Mémoire — évite crash OOM sur Windows si RAM < facteur de remplissage MUMPS

RISQUE : Aucun — 'AUTO' utilise in-core si possible, bascule out-of-core sinon

JUSTIFICATION : 'IN_CORE' est un hard-lock RAM : si la factorisation dépasse la mémoire disponible, Code_Aster plante au lieu de déborder proprement.

OPTIMISATION 2 — GESTION_MEMOIRE='AUTO' dans les cas ELUSIS

FICHER : aster/comm_blocks/railway_combination_inst.py, ligne ~764 (dans build_railway_seismic_results_block)

AVANT :

```
SOLVEUR=_F(  
    GESTION_MEMOIRE='IN_CORE',  
    METHODE='MUMPS',  
    NB_RHS={min(len(seismic_cases), 32)},  
    NPREC=6,  
    PCENT_PIVOT=20,  
    RENUM='METIS',  
    RESI_RELA=1e-03  
)
```

APRÈS :

```
SOLVEUR=_F(  
    GESTION_MEMOIRE='AUTO',  
    METHODE='MUMPS',  
    NB_RHS={min(len(seismic_cases), 32)},  
    NPREC=6,  
    PCENT_PIVOT=20,  
    RENUM='METIS',  
    RESI_RELA=1e-03  
)
```

GAIN ESTIMÉ : Mémoire — les 48 cas ELUSIS (NB_RHS=32+16) sont les plus lourds en RAM

RISQUE : Aucun

JUSTIFICATION : 48 RHS MUMPS simultanés avec 'IN_CORE' est le scénario qui cause le plus souvent les OOM sur cette chaîne.

OPTIMISATION 3 — DETRUIRE `deplcomb{i}*` immédiatement après chaque CREA RESU

Les champs assemblés (deplcomb{i}ELU, deplcomb{i}ELSCARA, deplcomb{i}ELSQPc, deplcomb{i}ELSQPf, deplcomb{i}ELSQP) ne sont utilisés **qu’une seule fois** : en CHAM_GD du CREA RESU qui les suit immédiatement.

FICHER : aster/comm_blocks/railway_combination_inst.py

Fonctions concernées : `build_railway_results_block_elu`,
`build_railway_results_block_elscara`, `build_railway_results_block_elsqp`

AVANT — fin du bloc par cas dans build railway results block élu (~ligne 1702):

```
lines.append(
f""""res{case.i}ELU = CREA_RESU(identifier='{current_identifrier}:1',
      AFFE= F(CARA_ELEM=elemprop,
```

```

        CHAM_GD=deplcomb{case.i}ELU,
        CHAM_MATER=fieldmat,
        INST=({inst_val:.1f}, ),
        MODELE=model),
    NOM_CHAM='DEPL ',
    OPERATION='AFFE ',
    TYPE_RESU='EVOL_ELAS')
"""
)
current_identifiant += 1
lines.append("")

APRÈS — ajouter DETRUIRE juste après CREA_RESU :

lines.append(
f"""res{case.i}ELU = CREA_RESU(identifiant='{current_identifiant}:1',
    AFFE=_F(CARA_ELEM=elemprop,
        CHAM_GD=deplcomb{case.i}ELU,
        CHAM_MATER=fieldmat,
        INST=({inst_val:.1f}, ),
        MODELE=model),
    NOM_CHAM='DEPL ',
    OPERATION='AFFE ',
    TYPE_RESU='EVOL_ELAS')

DETRUIRE(CONCEPT=_F(NOM=deplcomb{case.i}ELU))
"""
)
current_identifiant += 1
lines.append("")

```

Même modification à appliquer dans :

- build_railway_results_block_elscara → libère deplcomb{i}ELSCARA
- build_railway_results_block_elsqp → libère deplcomb{i}ELSQP,
deplcomb{i}ELSQPc, deplcomb{i}ELSQPf

GAIN ESTIMÉ : Mémoire — libère $N_{\text{cas}} \times 1$ champ NOEU_DEPL_R immédiatement après usage. Pour 30 cas sur un maillage 5000 nœuds : ≈ 300 MB libérés progressivement pendant les phases 7 et 9.

RISQUE : Aucun — les deplcomb{i}* ne sont plus référencés nulle part après leur CREA_RESU respectif.

JUSTIFICATION : Chaque CREA_CHAMP_ASSE crée un nouveau champ temporaire ; le conserver en base est du gaspillage pur après l'encapsulation EVOL_ELAS.

OPTIMISATION 4 — DETRUIRE des champs par cas après les CREA_RESU finaux

Après les blocs CREA_RESU finaux (ELU/DEPU/SIGU ; ELSCARA/DEPCR/SIGCR ; ELSQP/DEPQP/SIGQP ; ELUSIS/DEPSIS/SIGSIS), les champs individuels par cas (fca1ELU{i}, DEPL{i}ELU, sigELU{i}, ...) ne sont **plus jamais utilisés**.

FICHER : aster/comm_blocks/railway_combination_inst.py

Fonctions concernées : build_railway_crea_resu_elu_block,
build_railway_crea_resu_elscara_block, build_railway_crea_resu_elsqp_block,
build_railway_crea_resu_elusis_block

AVANT — fin de build_railway_crea_resu_elu_block (~ligne 2328) :

```
return "\n".join(lines).rstrip(), current_identifieur
```

APRÈS — ajouter le bloc DETRUIRE avant le return :

```
# Libération des champs par cas après assemblage dans ELU / DEPU / SIGU
all_noms = (
    [f"fcalELU{case.i}" for case in cases]
    + [f"DEPL{case.i}ELU" for case in cases]
    + [f"sigELU{case.i}" for case in cases]
)
concept_list = ", ".join(f"_F(NOM={n})" for n in all_noms)
lines.append(f"\nDETRUIRE(CONCEPT=({concept_list}))\n")

return "\n".join(lines).rstrip(), current_identifieur
```

Même modification pour :

- build_railway_crea_resu_elscara_block → libère fc~~al~~ELSCARA{i},
DEPL{i}ELSCARA, sigELSCARA{i}
- build_railway_crea_resu_elsqp_block → libère fc~~al~~ELSQP{i}, DEPL{i}ELSQP,
sigELSQP{i}
- build_railway_crea_resu_elusis_block → libère fc~~al~~ELUSIS{i}, DEPL{i}ELUSIS,
sigELUSIS{i}

GAIN ESTIMÉ : Mémoire + Temps — libère $N_{\text{cas}} \times 3$ champs par famille avant
CALC_FERRAILLAGE. Pour $30 \text{ cas} \times 4 \text{ familles} \times 3 \text{ types} = \mathbf{360 \text{ champs éliminés}}$
avant la phase ferrailage.

RISQUE : Faible — vérifier que ces champs ne sont pas exportés dans IMPR_RESU (ils ne
le sont pas : dalot_phase11.comm n'exporte que ELU, DEPU, SIGU, ELSQP, etc.)

JUSTIFICATION : La phase 11 accumule tous les objets intermédiaires jusqu'à
IMPR_RESU. La libération progressive réduit le pic mémoire avant
CALC_FERRAILLAGE, qui est lui-même coûteux.

Récapitulatif

Parties laissées intactes

Élément	Raison
LIRE_MAIILLAGE, MODI_MAIILLAGE, AFFE_MODELE, AFFE_CARA_ELEM, AFFE_MATERIAU	Zone intangible
Toutes les DEFI_FONCTION (PTmin/max, PL, loadR)	Zone intangible

Élément	Raison
Tous les chargements (load0, loadP, loadC, loadPPrail, loadPL_, loadPT)	Zone intangible
Tous les MECA_STATIQUE cas unitaires (UCD*, UCR*) — EXCIT, noms, identifiants	Zone intangible — seul GESTION_MEMOIRE modifié dans le SOLVEUR
Tous les CALC_CHAMP (Uc*, UCcr*)	Zone intangible
Toutes les extractions CREA_CHAMP OPERATION='EXTR'	Zone intangible
Toutes les superpositions CREA_CHAMP OPERATION='ASSE'	Zone intangible — DETRUIRE ajouté après , pas à l'intérieur
Tous les coefficients (1.35 / 1.5 / 1.0 / ψ 2...)	Zone intangible
Tous les assemblages CREA_RESU finaux (ELU, ELSQP, etc.)	Zone intangible — DETRUIRE ajouté après
Tous les IMPR_RESU	Zone intangible
Fonction legacy build_railway_results_block (MECA_STATIQUE par combinaison)	Code mort, logique fonctionnelle existante

Modifications proposées

#	Fichier	Ligne(s)	Changement	Impact
OPT 1	railway_unit_cases.py	128	'IN_CORE' → 'AUTO'	Stabilité mémoire
OPT 2	railway_combination_inst.py	~764	'IN_CORE' → 'AUTO'	Stabilité mémoire ELUSI
OPT 3	railway_combination_inst.py	~1714, ~1773, ~1843/1887/1923	DETRUIRE deplcomb{i}* après CREA_RESU	-300 MB phase 7/9
OPT 4	railway_combination_inst.py	~2328, ~2369, ~2409, ~2451	DETRUIRE fcal*/DEPL*/sig* après CREA_RESU finaux	-360 champs phase 1

Aucune modification n'a été appliquée. En attente de confirmation.